

Strict Typist Profile (STP) - Usage Guide

Overview

The **Strict Typist Profile (STP)** is a comprehensive type safety and struct integrity analysis tool for the Binary Decomposition Interface (BDI) CRRSS system. STP provides deep analysis of type safety issues, struct layout problems, and casting safety concerns in C code.

Features

1. Type Validation Engine

- **Type Mismatch Detection:** Identifies incompatible type assignments
- **Implicit Conversion Detection:** Finds potentially unsafe implicit type conversions
- **Signed/Unsigned Mix Detection:** Catches dangerous mixed sign comparisons
- **Pointer Type Compatibility:** Validates pointer type assignments
- **Type Punning Detection:** Identifies potentially undefined behavior from type punning

2. Struct Alignment Analyzer

- **Struct Padding Analysis:** Identifies inefficient struct layouts with excessive padding
- **Alignment Issue Detection:** Finds misaligned struct members
- **Unaligned Access Detection:** Catches potentially slow or unsafe unaligned memory access
- **Struct Packing Analysis:** Warns about potentially problematic packed structs
- **Member Ordering Optimization:** Suggests better member ordering to reduce padding
- **Portability Issue Detection:** Identifies platform-specific struct layout problems

3. Type Casting Safety Checker

- **Unsafe Cast Detection:** Identifies potentially dangerous type casts
- **Narrowing Conversion Detection:** Finds casts that may lose data
- **Pointer Cast Safety:** Validates pointer type casts
- **Const-Correctness Checking:** Ensures const qualifiers are not violated
- **Integer Overflow Detection:** Identifies casts that may cause integer overflow

Quick Start

Basic Usage

```
#include "stp/stp.h"

// Initialize STP
stp_config_t config = {0};
config.strictness_level = STP_STRICTNESS_STRICT;
config.check_type_mismatches = true;
config.check_implicit_conversions = true;
config.check_struct_padding = true;
config.check_unsafe_casts = true;

stp_context_t* ctx = stp_initialize(&config);

// Analyze a source file
stp_issue_t issues[100];
uint32_t num_issues = 0;

crrss_status_t status = stp_analyze_file(
    ctx,
    "path/to/source.c",
    issues,
    100,
    &num_issues
);

// Process results
for (uint32_t i = 0; i < num_issues; i++) {
    printf("%s:%u: %s\n",
        issues[i].file_path,
        issues[i].line_number,
        issues[i].description);
}

// Generate report
stp_report_t report;
stp_generate_report(ctx, &report);
printf("Type Safety Score: %.2f\n", report.type_safety_score);

// Cleanup
stp_shutdown(ctx);
```

Configuration

Strictness Levels

STP supports four strictness levels:

1. **STP_STRICTNESS_PERMISSIVE** (Level 0)
 - Allows most type conversions
 - Minimal checking
 - Good for legacy code
2. **STP_STRICTNESS_MODERATE** (Level 1)
 - Warns on potentially unsafe operations

- Balanced approach
- Recommended for most projects

3. **STP_STRICTNESS_STRICT** (Level 2)

- Strict type checking
- Enforces good practices
- Recommended for safety-critical code

4. **STP_STRICTNESS_PARANOID** (Level 3)

- Maximum type safety enforcement
- All checks enabled
- Use for highest assurance requirements

Detailed Configuration Options

```
typedef struct {
    // Strictness level
    stp_strictness_level_t strictness_level;

    // Type validation options
    bool check_type_mismatches;           // Check for type incompatibilities
    bool check_implicit_conversions;      // Find implicit conversions
    bool check_signed_unsigned_mix;       // Detect mixed sign operations
    bool check_pointer_type_compat;       // Validate pointer compatibility
    bool check_type_punning;              // Identify type punning

    // Struct analysis options
    bool check_struct_padding;             // Analyze struct padding
    bool check_struct_alignment;           // Check alignment issues
    bool check_unaligned_access;           // Detect unaligned access
    bool check_struct_packing;             // Warn about packed structs
    bool check_member_ordering;            // Suggest ordering optimizations
    bool check_portability;                // Find portability issues

    // Type casting options
    bool check_unsafe_casts;               // Detect unsafe casts
    bool check_narrowing_conversions;      // Find narrowing conversions
    bool check_pointer_casts;              // Check pointer casts
    bool check_const_correctness;          // Validate const usage
    bool check_integer_overflow_casts;     // Detect overflow in casts

    // Analysis options
    bool generate_reports;                 // Enable report generation
    bool suggest_fixes;                    // Provide fix suggestions
    bool check_c23_compliance;             // Check C23 standard compliance
    const char* report_output_dir;         // Report output directory

    // Integration
    bool integrate_with_bpme;              // Integrate with BPME
    bool integrate_with_sciv;              // Integrate with SCIV
    bool integrate_with_msm;               // Integrate with MSM
} stp_config_t;
```

API Reference

Initialization Functions

stp_initialize()

```
stp_context_t* stp_initialize(const stp_config_t* config);
```

Creates and initializes an STP context with the given configuration.

Parameters:

- `config` : Configuration structure

Returns:

- Initialized context or NULL on failure

stp_shutdown()

```
void stp_shutdown(stp_context_t* ctx);
```

Cleans up and frees all resources associated with the STP context.

stp_reset()

```
crrss_status_t stp_reset(stp_context_t* ctx);
```

Resets the STP analysis state while keeping the configuration.

Type Validation Functions

stp_detect_type_mismatches()

```
crrss_status_t stp_detect_type_mismatches(  
    stp_context_t* ctx,  
    const char* file_path,  
    stp_issue_t* issues,  
    uint32_t max_issues,  
    uint32_t* num_issues  
);
```

Detects type mismatches in the given source file.

stp_detect_implicit_conversions()

```
crrss_status_t stp_detect_implicit_conversions(  
    stp_context_t* ctx,  
    const char* file_path,  
    stp_issue_t* issues,  
    uint32_t max_issues,  
    uint32_t* num_issues  
);
```

Finds implicit type conversions that may lose data or change semantics.

stp_detect_signed_unsigned_mix()

```

crrss_status_t stp_detect_signed_unsigned_mix(
    stp_context_t* ctx,
    const char* file_path,
    stp_issue_t* issues,
    uint32_t max_issues,
    uint32_t* num_issues
);

```

Detects comparisons or operations mixing signed and unsigned types.

Struct Analysis Functions

stp_analyze_struct_layout()

```

crrss_status_t stp_analyze_struct_layout(
    stp_context_t* ctx,
    const char* struct_name,
    const char* file_path,
    struct_layout_t* layout
);

```

Analyzes the memory layout of a struct.

Returns struct_layout_t with:

- Total size and useful size
- Padding bytes and percentage
- Member information
- Optimization suggestions

stp_detect_struct_padding_issues()

```

crrss_status_t stp_detect_struct_padding_issues(
    stp_context_t* ctx,
    const char* file_path,
    stp_issue_t* issues,
    uint32_t max_issues,
    uint32_t* num_issues
);

```

Identifies structs with suboptimal padding.

Type Casting Functions

stp_detect_unsafe_casts()

```

crrss_status_t stp_detect_unsafe_casts(
    stp_context_t* ctx,
    const char* file_path,
    stp_issue_t* issues,
    uint32_t max_issues,
    uint32_t* num_issues
);

```

Detects potentially unsafe type casts.

stp_analyze_cast_safety()

```
crrss_status_t stp_analyze_cast_safety(
    stp_context_t* ctx,
    const type_info_t* source_type,
    const type_info_t* target_type,
    type_conversion_t* conversion
);
```

Analyzes the safety of a specific type conversion.

Comprehensive Analysis**stp_analyze_file()**

```
crrss_status_t stp_analyze_file(
    stp_context_t* ctx,
    const char* file_path,
    stp_issue_t* issues,
    uint32_t max_issues,
    uint32_t* num_issues
);
```

Performs comprehensive type safety analysis on a file.

stp_analyze_directory()

```
crrss_status_t stp_analyze_directory(
    stp_context_t* ctx,
    const char* dir_path,
    stp_report_t* report
);
```

Analyzes all source files in a directory recursively.

Reporting Functions**stp_generate_report()**

```
crrss_status_t stp_generate_report(
    stp_context_t* ctx,
    stp_report_t* report
);
```

Generates a comprehensive analysis report.

stp_export_report()

```
crrss_status_t stp_export_report(
    stp_context_t* ctx,
    const stp_report_t* report,
    const char* output_path,
    const char* format
);
```

Exports a report to a file in the specified format.

Supported formats:

- "text": Human-readable text format
- "json": JSON format (future)
- "html": HTML format (future)

stp_calculate_safety_score()

```
crss_status_t stp_calculate_safety_score(
    stp_context_t* ctx,
    double* score
);
```

Calculates an overall type safety score (0.0-1.0).

Examples

Example 1: Detecting Type Mismatches

```
#include "stp/stp.h"

int main(void) {
    // Configure STP for type checking
    stp_config_t config = {0};
    config.strictness_level = STP_STRICTNESS_STRICT;
    config.check_type_mismatches = true;

    stp_context_t* ctx = stp_initialize(&config);

    // Analyze file
    stp_issue_t issues[50];
    uint32_t num_issues = 0;

    stp_detect_type_mismatches(
        ctx,
        "myfile.c",
        issues,
        50,
        &num_issues
    );

    // Report findings
    printf("Found %u type mismatches:\n", num_issues);
    for (uint32_t i = 0; i < num_issues; i++) {
        printf("  %s:%u - %s\n",
            issues[i].file_path,
            issues[i].line_number,
            issues[i].description);
        printf("    Recommendation: %s\n", issues[i].recommendation);
    }

    stp_shutdown(ctx);
    return 0;
}
```

Example 2: Struct Layout Optimization

```
#include "stp/stp.h"

void optimize_struct(const char* file, const char* struct_name) {
    stp_config_t config = {0};
    config.strictness_level = STP_STRICTNESS_MODERATE;
    config.check_struct_padding = true;

    stp_context_t* ctx = stp_initialize(&config);

    // Analyze struct layout
    struct_layout_t layout;
    stp_analyze_struct_layout(ctx, struct_name, file, &layout);

    printf("Struct: %s\n", layout.struct_name);
    printf("  Total size: %zu bytes\n", layout.total_size);
    printf("  Useful size: %zu bytes\n", layout.useful_size);
    printf("  Padding: %zu bytes (%.1f%%)\n",
        layout.padding_bytes,
        layout.padding_percentage * 100.0);

    if (layout.optimization_suggestion) {
        printf("  Suggestion: %s\n", layout.optimization_suggestion);
    }

    stp_shutdown(ctx);
}
```

Example 3: Comprehensive Code Analysis

```
#include "stp/stp.h"

void analyze_project(const char* dir_path) {
    // Configure for comprehensive analysis
    stp_config_t config = {0};
    config.strictness_level = STP_STRICTNESS_PARANOID;
    config.check_type_mismatches = true;
    config.check_implicit_conversions = true;
    config.check_signed_unsigned_mix = true;
    config.check_struct_padding = true;
    config.check_unsafe_casts = true;
    config.check_const_correctness = true;
    config.generate_reports = true;

    stp_context_t* ctx = stp_initialize(&config);

    // Analyze entire directory
    stp_report_t report;
    stp_analyze_directory(ctx, dir_path, &report);

    // Display summary
    printf("=== Type Safety Analysis Report ===\n");
    printf("Type Safety Score: %.2f\n", report.type_safety_score);
    printf("Overall Risk: %s\n",
           risk_level_to_string(report.overall_risk));
    printf("\nStatistics:\n");
    printf("  Files analyzed: %u\n", report.statistics.files_analyzed);
    printf("  Total issues: %u\n", report.statistics.total_issues_found);
    printf("  Critical: %u\n", report.statistics.critical_issues);
    printf("  High priority: %u\n", report.statistics.high_priority_issues);
    printf("  Medium priority: %u\n", report.statistics.medium_priority_issues);
    printf("  Low priority: %u\n", report.statistics.low_priority_issues);

    // Export detailed report
    stp_export_report(ctx, &report, "stp_report.txt", "text");

    stp_shutdown(ctx);
}
```

Issue Types

Type Validation Issues

- `STP_ISSUE_TYPE_MISMATCH` : Type mismatch in assignment or comparison
- `STP_ISSUE_IMPLICIT_CONVERSION` : Implicit type conversion
- `STP_ISSUE_SIGNED_UNSIGNED_MIX` : Mixing signed and unsigned types
- `STP_ISSUE_POINTER_TYPE_INCOMPAT` : Incompatible pointer types
- `STP_ISSUE_TYPE_PUNNING` : Type punning through unions or casts

Struct Issues

- `STP_ISSUE_STRUCT_PADDING` : Excessive struct padding
- `STP_ISSUE_STRUCT_ALIGNMENT` : Struct alignment problems
- `STP_ISSUE_UNALIGNED_ACCESS` : Unaligned memory access
- `STP_ISSUE_STRUCT_PACKING` : Potentially problematic packed struct

- `STP_ISSUE_MEMBER_ORDERING` : Suboptimal member ordering
- `STP_ISSUE_PORTABILITY` : Platform portability issues

Casting Issues

- `STP_ISSUE_UNSAFE_CAST` : Potentially unsafe cast
- `STP_ISSUE_NARROWING_CONVERSION` : Narrowing conversion (data loss)
- `STP_ISSUE_POINTER_CAST_UNSAFE` : Unsafe pointer cast
- `STP_ISSUE_CONST_VIOLATION` : Const qualifier violation
- `STP_ISSUE_INTEGER_OVERFLOW_CAST` : Potential integer overflow in cast

Integration with Other CRRSS Modules

STP can be integrated with other CRRSS modules for enhanced analysis:

```
// Initialize all modules
stp_context_t* stp_ctx = stp_initialize(&stp_config);
void* bpme_ctx = /* BPME context */;
void* sciv_ctx = /* SCIV context */;
void* msm_ctx = /* MSM context */;

// Integrate STP with other modules
stp_integrate_bpme(stp_ctx, bpme_ctx);
stp_integrate_sciv(stp_ctx, sciv_ctx);
stp_integrate_msm(stp_ctx, msm_ctx);

// Cross-module analysis provides deeper insights
```

Best Practices

1. **Start with Moderate Strictness:** Begin with `STP_STRICTNESS_MODERATE` and increase as needed
2. **Enable Relevant Checks:** Only enable checks relevant to your project's requirements
3. **Iterative Analysis:** Run STP regularly during development, not just at the end
4. **Fix High Priority Issues First:** Address critical and high-priority issues before others
5. **Use Reports:** Generate and review reports to track progress over time
6. **Integrate in CI/CD:** Add STP checks to your continuous integration pipeline

Performance Considerations

- **File Size:** STP performs best on files under 10,000 lines
- **Memory Usage:** Approximately 1MB per 1000 issues tracked
- **Analysis Speed:** ~1000 lines/second on modern hardware
- **Batch Processing:** Use `stp_analyze_directory()` for better performance on multiple files

Limitations

- **Pattern-Based Detection:** Current implementation uses pattern matching; full AST analysis is planned
- **C23 Support:** Full C23 feature support is in progress
- **Cross-File Analysis:** Limited cross-translation-unit analysis capability

- **Macro Expansion:** Macros are not fully expanded during analysis

Future Enhancements

- Full AST-based analysis using Clang/LibTooling
- Machine learning-based issue prioritization
- IDE integration (VSCode, CLion, Vim)
- Real-time analysis during editing
- Auto-fix capabilities for common issues

Support and Contribution

For bug reports, feature requests, or contributions:

- GitHub: [Binary Decomposition Interface Repository](https://github.com/Mecca-Research/The-Binary-Decomposition-Interface) (<https://github.com/Mecca-Research/The-Binary-Decomposition-Interface>)
- Documentation: See `tools/crrss/docs/`
- Tests: See `tools/crrss/tests/test_stp.c`

License

Part of the Binary Decomposition Interface (BDI) project.
See LICENSE file for details.